

Programmation Fonctionnelle (YSINL4B1)
1 heure

Tout document (électronique ou papier) est interdit. L'accès à internet, les ordinateurs portables, téléphones portables, montres, clés usb et plus généralement tout matériel électronique sont interdits.

Consignes :

- Toutes les fonctions définies pour les questions de l'examen doivent être signées.
-

1. Évaluer les expressions suivantes en justifiant votre résultat :
 - (a) `map (* 2) [1..5]`
 - (b) `product (filter even [1..5])`
 - (c) `foldl (+) 0 [1..5]`
2. Quelle est la signature (type) des fonctions définies ci-dessous ? Justifier votre réponse en indiquant ce qu'elles calculent ?
 - (a) `f1 :: ..... ?`
`f1 xs = [x | x <- xs, mod x 2 == 0]`
 - (b) `f2 :: ..... ?`
`f2 xs = or [mod x 2 == 0 | x <- xs]`
 - (c) `f3 :: ..... ?`
`f3 xs = and [mod x 2 == 0 | x <- xs]`
3. Définir la fonction (`insere x ys`) qui insère un élément `x` à sa place dans une liste `ys` ordonnée par ordre croissant. Votre définition doit utiliser la récursivité avec le filtrage par motifs (pattern-matching).
`insere 3 [1..8] ==> [1, 2, 3, 3, 4, 5, 6, 7, 8]`
`insere 12 [1, 2, 7, 15, 23] ==> [1, 2, 7, 12, 15, 23]`
`insere 'q' "abchprs" ==> "abchpqrs"`
4. En utilisant une ZF-expression (liste en compréhension), définir la fonction (`intersect xs ys`) qui calcule l'intersection de deux listes `xs` et `ys`.
`intersect [1, 5, 6] [7, 6, 9, 1] ==> [1, 6]`
`intersect [1, 5, 6] [7, 2, 9] ==> []`
`intersect "abcd" "gecfa" ==> "ac"`
5. Dans cette question, on considère un dépôt de produits. Chaque produit est caractérisé par trois attributs : une chaîne de caractères `refP` identifiant le produit de manière unique, une chaîne de caractères `typeP` décrivant le type du produit, un nombre réel `prix` représentant le prix unitaire du produit.
 - (a) Définir un nouveau type nommé `Produit` dont les valeurs représentent les produits du dépôt. Chaque valeur `Produit` doit contenir les trois attributs décrits ci-dessus nommés `refP`, `typeP` et `prix`.

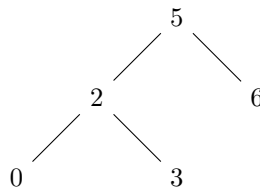
- (b) Définir une fonction `prixTotal` qui prend deux paramètres en entrée : (i) une valeur de type `Produit` et (ii) un nombre réel positif représentant la quantité commandée de ce produit. La fonction `prixTotal` doit calculer et retourner le prix total de ce produit.

```
prixTotal (Produit "CH262" "Cheville" 75) 30 ==> 2250.0
prixTotal (Produit "CL45" "Cheville" 105) 10 ==> 1050.0
```

6. Un Arbre Binaire de Recherche (ABR) est un arbre binaire tel que, pour tout nœud portant une valeur x , toutes les valeurs portées par son fils gauche sont inférieures à x , et toutes les valeurs portées par son fils droit sont supérieures à x . Il est important de noter qu'un arbre peut être vide. Nous utilisons le type suivant pour représenter un tel arbre :

```
data ABR a = Vide | Noeud a (ABR a) (ABR a) deriving (Show)
```

- (a) Coder l'arbre suivant, nommé `abr`, en utilisant le nouveau type `ABR`.



- (b) Définir la fonction (`sommeABR t`) qui calcule la somme des éléments d'un arbre `ABR t` étiqueté par des entiers.

```
sommeArbre abr ==> 16
```

- (c) Définir la fonction (`insereABR e t`) qui prend en paramètres un élément `e` et un arbre `ABR` et retourne un arbre `ABR` où l'élément `e` est inséré à sa bonne place dans `t`.

